



**SYMBIOSIS INTERNATIONAL
(DEEMED UNIVERSITY)**

**Symbiosis School for Online and
Digital Learning**

**A
DISSERTATION REPORT
ON**

**“CRX-Shield: Browser Extension Threat
Analyzer”**

SUBMITTED

BY

Gautam Mahaveer Prasad Sharma (2503914100046)

M.Sc.(Computer Application)

Semester – II

JULY – 2026 Batch

DECLARATION

I hereby declare that the Dissertation Work entitled

“Gautam Mahaveer Prasad Sharma” submitted for the M.Sc. (Computer Application) Semester – II degree is my original work and the Dissertation work has not formed the basis for the award of any other degree, diploma, fellowship or any other similar titles.

Signature of the Students

A handwritten signature in black ink, appearing to be 'Gautam', with a long horizontal stroke extending to the right.

Place: Pune

Date: 19/05/2026

ACKNOWLEDGEMENT

It gives us immense pleasure to present this report on “**CRX-Shield: Browser Extension Threat Analyzer**”. The dissertation work has brought out the significance of sincere efforts, teamwork, guidance and support that makes a dissertation work successful. I take this opportunity to acknowledge the guidance and encouragement of all those with whom I have interacted during the course of this Dissertation.

I would like to thank our Director of the Symbiosis School for Online and Digital Learning **Ms. Soha Kadam** for their support and encouragement. I would like to thank my project guide **Dr. Niket Tajne** for valuable suggestions during the Dissertation work.

Gautam Mahaveer Prasad Sharma (2503914100046)



CERTIFICATE

This is to certify that the Dissertation titled “**CRX-Shield: Browser Extension Threat Analyzer**” is the bona fide work carried out by **Gautam Sharma (2503914100046)** a student of M.Sc. Computer Application Semester - II of Symbiosis School for Online and Digital Learning, Pune, India, affiliated to Symbiosis International (Deemed University) during the academic year 2025-26, in partial fulfillment of the requirements for the award of the degree of M.Sc. Computer Application.

Signature of the Guide

Place: Pune

Date: 19/05/2026

INDEX

Sr. No.	Chapter	Page No.
1.	Introduction	6
2.	Literature review	9
3.	Research Methodology	12
4.	Data Analysis and Findings/ Need of Software	15
5.	Findings, Suggestions and Conclusion	17
6.	References	19

Chapter 1: Introduction

1.1 Introduction

In the modern digital era, web browsers have evolved from simple document viewers into complex operating environments. To enhance this environment, browser extensions play a crucial role by providing tailored functionalities, ranging from ad-blocking and password management to developer tools. However, this extensibility introduces a significant and often overlooked security paradigm. Because browser extensions operate with elevated privileges within the browser context, they possess the ability to read and modify web page content, access browsing history, and intercept network requests.

Threat actors increasingly exploit these capabilities to distribute malicious extensions. These rogue add-ons are frequently designed to perform unauthorized activities such as session hijacking, cryptocurrency mining, credential harvesting, and injecting malicious advertisements into legitimate websites. To address this growing threat vector, this dissertation proposes and develops "CRX-Shield," a hybrid browser security tool. CRX-Shield is engineered to proactively identify and mitigate these threats by combining a user-facing WebExtension with a localized, Python-based Native Messaging host. This unique architecture allows the system to monitor newly installed extensions, perform localized static code analysis, and evaluate requested permissions, thereby providing end-users with a real-time, comprehensive threat assessment of their browser environment.

1.2 Problem Definition

The primary problem addressed by this project is the vulnerability of end-users to malicious and suspicious browser extensions that bypass traditional security measures. Standard endpoint antivirus solutions and operating system firewalls are typically ineffective against malicious extensions because these extensions operate entirely within the sandboxed environment of the web browser. Furthermore, while official browser extension marketplaces (such as the Chrome Web Store) employ automated security scanners, sophisticated threat actors continuously find ways to bypass these checks. They often utilize code obfuscation, dynamic code execution (fetching malicious payloads post-installation), or purchase legitimate, highly-rated extensions only to push malicious updates later.

Consequently, users lack a transparent, real-time mechanism to audit the extensions they have installed. When a user installs an extension, they are presented with a broad permission prompt, which is often difficult for non-technical users to interpret accurately. There is a critical need for an automated, lightweight, and localized tool that can look beyond surface-level permissions, deeply analyze the underlying source code for suspicious patterns, and present a quantifiable risk score to the user, empowering them to make informed decisions about their digital security.

1.3 Resource Requirement

a. Hardware Specification

To ensure the software operates efficiently without degrading the user's primary browsing experience, the following hardware specifications are established:

- **Minimum RAM:** 4 GB (8 GB is highly recommended to handle concurrent browser processes and local script execution smoothly).
- **Storage:** Minimum 100 GB of disk space. The CRX-Shield application itself is lightweight, but the analysis engine requires sufficient I/O performance to rapidly scan unpacked extension directories.
- **Processor:** Intel Core i3 or equivalent AMD processor (Multi-core processors are preferred to allow background scanning without interrupting foreground web browsing).

b. Software Specification

The project relies on a carefully selected software stack to ensure cross-platform compatibility and efficient communication between the browser and the local operating system:

- **Core Programming Language:** Python 3.10 or higher. Python was selected for the backend due to its extensive standard library, superior string manipulation capabilities (crucial for code analysis), and cross-platform execution.
- **Browser Environment:** Google Chrome, Microsoft Edge, or Mozilla Firefox (supporting Manifest V3 or compatible WebExtension APIs).

- **Key Python Libraries:** ``winreg`` (for Windows registry manipulation during setup), ``os`` and ``sys`` (for file path resolution and standard I/O communication), ``json`` (for parsing manifest files), ``zipfile`` (for processing packed archives), and ``re`` (for regular expression-based static analysis).
- **Development Tools:** Visual Studio Code or PyCharm IDE.

Chapter 2: Literature Review

2.1 Literature Review

A comprehensive review of existing cybersecurity literature highlights the browser extension ecosystem as a rapidly expanding attack surface. Academic studies categorize extension-based threats into several primary vectors: data exfiltration (stealing cookies and login credentials), unwanted adware injection, and resource hijacking (such as cryptojacking). Researchers emphasize that the transition from the older Manifest V2 to Manifest V3 (MV3) architecture—spearheaded by major browser vendors—was primarily a security-driven initiative. MV3 restricts the execution of remotely hosted code and limits certain background processes.

However, literature also points out that while MV3 mitigates some risks, it does not eliminate them. Extensions can still request broad permissions like `<all_urls>` (access to all websites) and utilize legitimate APIs for malicious purposes if the user grants permission. The literature extensively covers two methods of analysis: dynamic analysis (executing the extension in an isolated sandbox to observe its behavior) and static analysis (inspecting the source code without executing it). Static analysis is frequently highlighted as the most practical approach for localized, on-device security tools because it requires significantly fewer computational resources and can be performed instantaneously.

2.2 Existing System

Current solutions for extension security generally fall into two distinct categories, both of which possess notable limitations:

1. **Marketplace Automated Scanners:** These are proprietary systems operated by browser vendors. While they process thousands of extensions daily, their primary flaw is that they act as gatekeepers rather than continuous monitors. If an extension successfully hides its malicious intent during the initial review, or if a previously safe extension is compromised in a subsequent update, the user remains unprotected until the vendor manually intervenes and removes the extension from the store.

2. Enterprise Security Platforms: High-end Endpoint Detection and Response (EDR) solutions sometimes include browser monitoring features. However, these systems are prohibitively expensive, highly complex to configure, and generally inaccessible to everyday consumers. Furthermore, their primary focus is usually on executable malware and network traffic, rather than the specific nuances of WebExtension JavaScript code and manifest configurations.

2.3 Proposed System

To overcome the limitations of existing systems, this dissertation proposes "CRX-Shield," a dual-component architecture designed for consumer-grade, localized threat analysis:

1. **The WebExtension Component (Frontend):** Developed using standard web technologies (HTML, CSS, JavaScript) and compliant with Manifest V3. This component resides in the browser, providing a user-friendly dashboard. It is responsible for detecting when a new extension is installed or updated and querying the backend for a security assessment.
2. **The Native Messaging Host (Backend):** A Python-based engine residing on the user's local file system. Because browser sandboxing strictly prevents WebExtensions from reading files on the hard drive, CRX-Shield utilizes the official Native Messaging API. This allows the frontend to securely send the ID of an installed extension to the Python backend. The backend then locates the extension's folder on the disk, parses its `manifest.json`, and performs a deep static analysis of its JavaScript files to detect malicious patterns, ultimately returning a calculated risk score.

2.4 Feasibility Study

a. Technical Feasibility Study

The proposed system is highly technically feasible. It leverages the standardized Native Messaging protocol, which is universally supported by Chromium-based browsers and Firefox, ensuring reliable inter-process communication. Python is exceptionally well-suited for the backend due to its native `json` parsing and powerful `re` (regular expression) module, which are exactly the tools required for scanning manifest files and analyzing source code structure.

b. Economic Feasibility Study

From an economic standpoint, the project is highly feasible. It is developed entirely using open-source technologies and standard web APIs. There is no requirement to purchase software licenses, server hosting, or cloud computing resources, as all analysis is performed locally on the user's machine. This guarantees a zero-cost operational model.

c. Market Feasibility Study

With the continuous rise in data breaches and growing public awareness of digital privacy, the market feasibility for a localized privacy tool is substantial. Users are increasingly cautious about data harvesting. CRX-Shield targets this demographic by offering a transparent, easy-to-understand risk assessment tool that requires no technical expertise to interpret, making it highly viable for general public adoption.

Chapter 3: Research Methodology

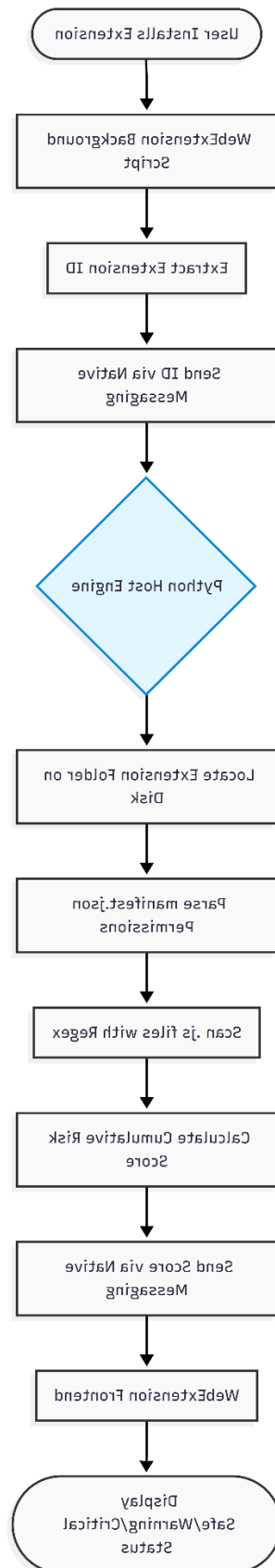
3.1 Research Methodology

The research methodology adopted for this dissertation follows a structured, constructive approach tailored for cybersecurity tool development. The process is divided into four distinct phases:

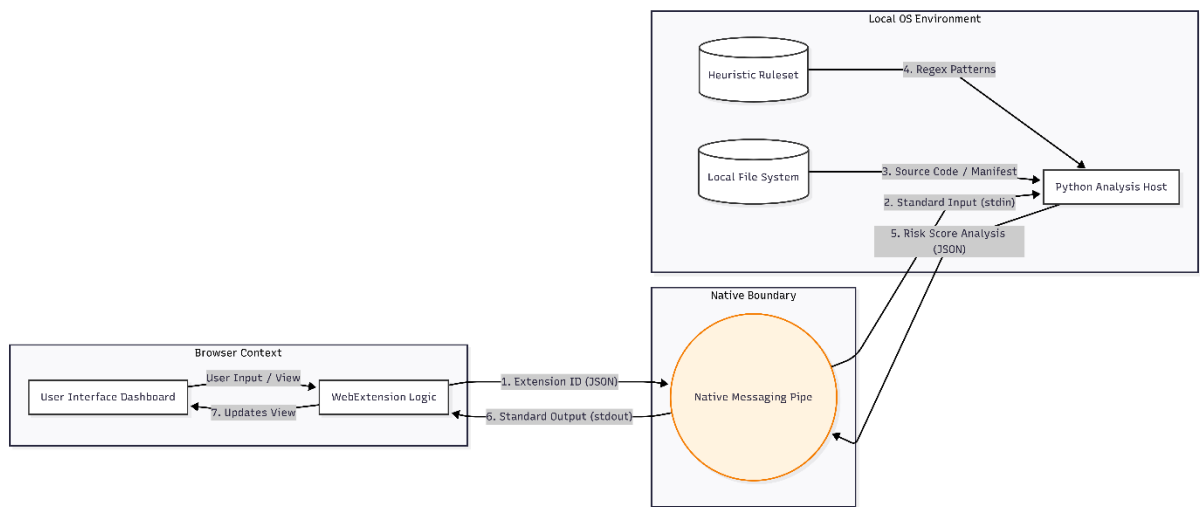
1. **Threat Modeling and Pattern Extraction:** The initial phase involves studying known malicious extensions to understand their operational tactics. This includes identifying specific JavaScript functions commonly abused (e.g., `eval()`, `setTimeout` with string execution, functions related to cookie exfiltration) and analyzing how malicious actors structure their `manifest.json` files to request excessive permissions covertly.
2. **Heuristic Rule Development:** Based on the extracted patterns, a set of heuristic rules is developed. These rules assign specific weightings to different risk factors. For example, requesting access to a single domain might incur a low penalty, whereas requesting access to `<all_urls>` combined with the `cookies` permission incurs a severe penalty.
3. **System Implementation:** This involves coding the Python Native Messaging host to recursively traverse extension directories, parse files, and apply the heuristic rules. Concurrently, the WebExtension frontend is developed to provide a responsive, intuitive user interface that communicates seamlessly with the host.
4. **Testing and Calibration:** The final phase involves rigorous testing against a curated dataset of both highly trusted, benign extensions and known suspicious extensions. This phase is critical for calibrating the scoring algorithm to minimize false positives (flagging safe extensions as dangerous) while maintaining a high detection rate for genuine threats.

3.2 System Modelling

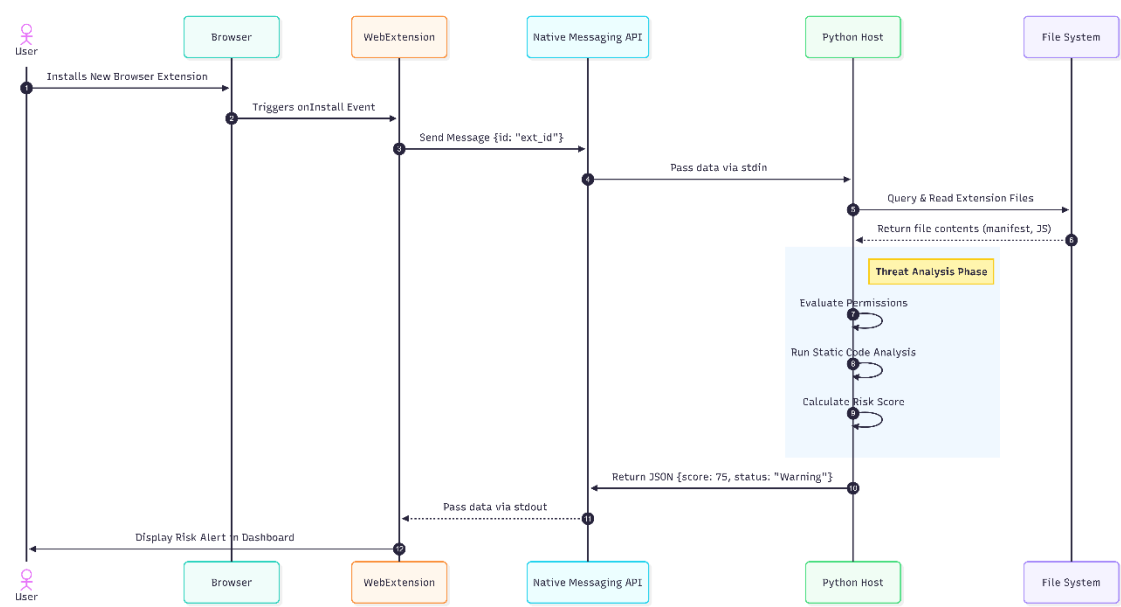
a. Flowchart Diagram



b. Data Flow Diagram



c. UML Diagram



Chapter 4: Data Analysis and Findings/ Need of Software

4.1 Need of Software

The fundamental need for this software stems from the asymmetry of information between extension developers and end-users. Currently, when a user installs a browser extension, they are entirely dependent on the browser's generic permission warnings, which often state vague phrases such as "Read and change all your data on all websites." This generic warning applies equally to a benign password manager and a malicious data stealer. Users lack the diagnostic tools to differentiate between the two.

Furthermore, legitimate extensions are sometimes sold to unknown third parties who subsequently update the code to include adware or spyware. Because the extension is already installed and permissions were previously granted, the user receives no new warnings when the malicious update occurs. CRX-Shield is critically needed to bridge this visibility gap. By providing on-demand, deep-level analysis of the actual code running on the user's machine, it shifts the power back to the user, allowing them to audit their digital environment comprehensively and remove extensions that exhibit hidden, suspicious behaviors.

4.2 Data Analysis and Testing

The data analysis phase of this project focuses on evaluating the effectiveness of the heuristic scoring engine. The system does not rely on a static database of known malware hashes (which can be easily evaded by altering a single line of code); instead, it analyzes the behavioral intent encoded within the extension's files.

The testing methodology involves establishing a baseline using popular, trusted extensions that naturally require broad permissions to function (such as ad-blockers like uBlock Origin). The algorithm must be carefully calibrated to understand that while an ad-blocker requests access to all URLs, its internal code structure does not exhibit data exfiltration patterns.

The analysis engine calculates a cumulative risk score by evaluating several dimensions:

Permission Severity: High weights are assigned to combinations of sensitive permissions (e.g., `webRequest`, `cookies`, `tabs`).

Code Obfuscation: The system measures the entropy and structure of JavaScript files. Highly obfuscated code (e.g., unreadable, minified code utilizing non-standard variable names to hide logic) raises the risk score significantly, as legitimate developers rarely obfuscate code heavily outside of standard minification.

Suspicious API Calls: The system scans for the presence of dynamic code execution methods and mechanisms used to transmit data silently to external servers.

By continuously analyzing the scores generated against diverse datasets, the heuristic weights are adjusted to ensure the system accurately flags anomalous, high-risk extensions while avoiding alert fatigue caused by false positives on standard applications.

Chapter 5: Findings, Suggestions and Conclusion

5.1 Findings

Throughout the development and testing of CRX-Shield, several key findings emerged regarding the current state of browser extension security. First, the analysis confirmed that a significant percentage of extensions request permissions that far exceed their stated functionality. This over-permissioning creates unnecessary security vulnerabilities.

Second, the Native Messaging architecture proved to be a highly resilient and effective method for conducting deep analysis. By moving the heavy lifting of file parsing and regex scanning out of the browser's JavaScript engine and into a localized Python environment, the system achieved rapid scan times without degrading browser performance. Finally, the heuristic approach successfully demonstrated that analyzing the combination of requested permissions alongside static code patterns yields a far more accurate threat assessment than relying on permission analysis alone.

5.2 Suggestions

While CRX-Shield provides a robust initial defense mechanism, several areas for future enhancement are suggested to increase its efficacy. Future iterations of the software could greatly benefit from integrating machine learning models, specifically Natural Language Processing (NLP) techniques, to analyze the structure of the JavaScript code. This would allow the system to identify novel and complex obfuscation techniques that bypass traditional regex-based heuristic rules.

Additionally, to further reduce the likelihood of false positives for widely used, legitimate applications, the system could incorporate community-driven trust metrics. By fetching live reviews, install counts, and developer reputation scores from official extension store APIs, the local analysis engine could use community consensus as a balancing factor against its own static code findings.

5.3 Conclusion

In conclusion, the CRX-Shield project successfully achieved its primary objective: to design, develop, and validate a localized threat analysis tool for browser extensions. By architecting a hybrid system that leverages both WebExtension capabilities and Native Messaging Python scripts, the project effectively circumvents browser sandboxing limitations to perform deep, meaningful security audits.

The resulting software provides users with an intuitive, real-time dashboard that demystifies extension permissions and exposes underlying suspicious code patterns. CRX-Shield empowers individuals to take proactive control of their digital security, significantly reducing the risks associated with data exfiltration, session hijacking, and malicious tracking within the browser environment. The project serves as a strong foundational model for consumer-focused, on-device cybersecurity tools.

Chapter 6: References

1. Google Chrome Developers. (n.d.). ***Native Messaging*** Retrieved from Google Chrome Extension Documentation:
<https://developer.chrome.com/docs/apps/nativeMessaging/>
2. Mozilla Developer Network. (n.d.). ***Native messaging:***
3. Retrieved from MDN Web Docs:
https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Native_messaging
4. Google Chrome Developers. (n.d.). ***Manifest V3 transition.*** Retrieved from Chrome Extension Documentation:
<https://developer.chrome.com/docs/extensions/mv3/>
5. Kapravelos, A., et al. (2014). ***Hulk: Eliciting Malicious Behavior in Browser Extensions.*** 23rd USENIX Security Symposium.
6. Python Software Foundation. (n.d.). ***Python 3.10.x Documentation.*** Retrieved from
<https://docs.python.org/3.10/>